

Q: What is a Database?

Answer: A systematic, organized collection of structured information or data, typically stored electronically in a computer system for easy access and management.

Q: What is a DBMS?

Answer: Database Management System (DBMS) is software acting as an interface between the user and the database to securely store, retrieve, and manage data (e.g., MySQL, Oracle).

Q: File System vs. DBMS?

Answer: File systems store data in flat files, leading to data redundancy, poor security, and difficult concurrent access. DBMS provides structured storage, ACID properties, indexing, and multi-user access.

Q: What is an RDBMS?

Answer: Relational Database Management System. It stores data in tabular format (rows and columns) and establishes relationships between tables using keys.

Q: DBMS vs. RDBMS?

Answer: DBMS stores data as files/navigational structures, doesn't enforce relationships, and is for smaller datasets. RDBMS stores data in tables, enforces relationships, and scales for enterprise data.

Q: What are the ACID properties?

Answer: Atomicity, Consistency, Isolation, and Durability—four key principles that guarantee database transactions are processed reliably.

Q: Explain Atomicity?

Answer: All or nothing. A transaction is treated as a single indivisible unit. If one part fails, the entire transaction fails and the database state is left unchanged.

Q: Explain Consistency?

Answer: Ensures that a transaction brings the database from one valid state to another, maintaining all predefined rules, constraints, and triggers.

Q: Explain Isolation?

Answer: Ensures that concurrent execution of multiple transactions leaves the database in the same state as if the transactions were executed sequentially.

Q: Explain Durability?

Answer: Once a transaction is committed, its changes are permanent and will survive system failures or power outages.

Q: What is Data Abstraction in DBMS?

Answer: The process of hiding complex internal data structures and storage details from the end-users to simplify their interaction with the database.

Q: What are the three levels of Data Abstraction?

Answer: Physical level (how data is stored), Logical level (what data is stored and their relationships), and View level (what specific users are allowed to see).

Q: What is Data Independence?

Answer: The ability to modify a schema definition in one level without affecting the schema definition in the next higher level.

Q: Physical vs. Logical Data Independence?

Answer: Physical independence means changing storage structures (like adding an index) doesn't affect the logical schema. Logical independence means changing table structures (like adding a column) doesn't break external user views.

Q: What is a Database Schema?

Answer: The overall logical design, structure, and constraints of the database. It is a blueprint that rarely changes.

Q: Schema vs. Instance?

Answer: Schema is the static blueprint of the database. An instance is the actual data (snapshot) contained in the database at a specific moment in time.

Q: What is a Data Model?

Answer: A collection of conceptual tools for describing data, data relationships, data semantics, and consistency constraints.

Q: What is the ER Model?

Answer: Entity-Relationship Model is a high-level conceptual data model representing real-world entities and their associations using ER diagrams.

Q: Entity vs. Attribute?

Answer: An Entity is a real-world object (e.g., Student). An Attribute is a property that describes the entity (e.g., Age, Name).

Q: What is a Weak Entity?

Answer: An entity that cannot be uniquely identified by its own

attributes alone and relies on a relationship with a strong entity (owner) to exist.

Q: What is a Relationship in ER?

Answer: An association among two or more entities (e.g., a Teacher instructs a Student).

Q: What are Cardinality Ratios?

Answer: They define the maximum number of relationship instances an entity can participate in (One-to-One, One-to-Many, Many-to-One, Many-to-Many).

Q: What is a Super Key?

Answer: A single attribute or a combination of attributes that can uniquely identify a row in a table.

Q: What is a Candidate Key?

Answer: A minimal Super Key. It is a set of attributes that uniquely identifies tuples, with no redundant attributes.

Q: What is a Primary Key?

Answer: A specific Candidate Key chosen by the database designer to uniquely identify records in a table. It cannot be NULL.

Q: What is a Foreign Key?

Answer: An attribute in one table that references the Primary Key of another table, used to establish and enforce a link between the data.

Q: What is an Alternate Key?

Answer: All Candidate Keys that were not selected to be the Primary Key.

Q: What is a Composite Key?

Answer: A primary key consisting of two or more attributes that together uniquely identify a record.

Q: Unique Constraint vs. Primary Key?

Answer: Both ensure uniqueness. However, a table can have multiple Unique constraints but only one Primary Key. Unique constraints allow one NULL value (in SQL Server/MySQL) while Primary Keys allow none.

Q: What is an Integrity Constraint?

Answer: Rules enforced on data columns to ensure the accuracy, validity, and consistency of the data in a database.

Q: What is Domain Integrity?

Answer: Ensures that all values in a column fall within a defined

domain (valid range, type, or format).

Q: What is Referential Integrity?

Answer: Ensures that the relationship between two tables remains synchronized during updates and deletes (enforced by Foreign Keys).

Q: What is an Anomaly in DBMS?

Answer: An inconsistency or flaw in a database that arises due to poor design, typically manifesting during insertion, deletion, or updating of data.

Q: Types of Anomalies?

Answer: Insertion Anomaly (cannot insert data without presence of other data), Deletion Anomaly (deleting a row unintentionally deletes other critical data), Update Anomaly (updating one row requires updating multiple rows).

Q: What is Functional Dependency (FD)?

Answer: A relationship where one attribute uniquely determines another attribute. If $A \rightarrow B$, knowing A means you know exactly one value of B.

Q: Trivial vs. Non-Trivial FD?

Answer: Trivial: $A \rightarrow B$ where B is a subset of A. Non-Trivial: $A \rightarrow B$ where B is not a subset of A.

Q: What is Normalization?

Answer: The process of organizing data to minimize redundancy and remove insert, update, and delete anomalies by breaking tables down into smaller, related tables.

Q: Why do we need Normalization?

Answer: To optimize storage space, maintain data consistency, and ensure logical data dependencies make sense.

Q: First Normal Form (1NF)?

Answer: A table is in 1NF if it contains only atomic (indivisible) values and has no repeating groups or arrays.

Q: Second Normal Form (2NF)?

Answer: It must be in 1NF and have no Partial Dependencies (every non-prime attribute must fully depend on the entire candidate key, not just a part of it).

Q: What is a Partial Dependency?

Answer: When a non-prime attribute is dependent on only a portion of a composite primary key.

Q: Third Normal Form (3NF)?

Answer: It must be in 2NF and have no Transitive Dependencies (a non-prime attribute should not depend on another non-prime attribute).

Q: What is a Transitive Dependency?

Answer: If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$ is a transitive dependency.

Q: Boyce-Codd Normal Form (BCNF)?

Answer: A stricter version of 3NF. For every non-trivial functional dependency $X \rightarrow Y$, X must be a super key.

Q: 3NF vs. BCNF?

Answer: BCNF handles situations where a table has overlapping candidate keys. Every BCNF is in 3NF, but not every 3NF is in BCNF.

Q: What is Denormalization?

Answer: The deliberate process of introducing redundancy into a database (combining tables) to improve read performance and reporting speed.

Q: When should we use Denormalization?

Answer: In Data Warehouses and analytics systems where complex joins slow down heavy read-operations, and data is updated infrequently.

Q: Relational Algebra vs. Relational Calculus?

Answer: Relational Algebra is a procedural query language (specifies how to get data). Relational Calculus is non-procedural (specifies what data to get).

Q: Selection operator?

Answer: Filters rows in a relation that satisfy a specific proposition/condition.

Q: Projection operator?

Answer: Extracts specific columns from a relation, removing duplicate tuples in the result.

Q: Cartesian Product?

Answer: Combines information from two relations, pairing every row of the first table with every row of the second.

Q: Union and Intersection?

Answer: Union combines tuples from two relations, removing duplicates. Intersection returns tuples present in both relations.

Q: What is a Join in Relational Algebra?

Answer: A Cartesian product followed by a selection condition. It

combines related tuples from two relations into single tuples.

Q: What is a Theta Join?

Answer: A join that combines tables based on any specific condition rather than just equality.

Q: What is an Equi Join?

Answer: A specific type of Theta join where the condition is strictly based on equality between columns.

Q: Natural Join?

Answer: An Equi Join that automatically matches all columns with the same name across two tables and removes duplicate columns in the result.

Q: What is an Outer Join?

Answer: A join that preserves tuples from one or both relations even if they do not have matching values in the other relation, filling blanks with NULLs.

Q: Left Outer Join vs. Right Outer Join?

Answer: Left Outer keeps all rows from the left table. Right Outer keeps all rows from the right table.

Q: Division Operator?

Answer: Used for queries containing the word all. E.g., Find the names of students who have taken all courses offered.

Q: What is a Tuple?

Answer: A single row or record in a relational database table.

Q: What is SQL?

Answer: Structured Query Language. The standard language used to communicate with, manage, and manipulate relational databases.

Q: SQL vs. PostgreSQL / Supabase?

Answer: SQL is the language itself. PostgreSQL is an advanced, robust object-relational database management system that executes SQL. Supabase is a backend-as-a-service built on top of PostgreSQL.

Q: What are the sub-languages of SQL?

Answer: DDL, DML, DQL, DCL, and TCL.

Q: What is DDL (Data Definition Language)?

Answer: Commands used to define database structures. Auto-committed. (CREATE, ALTER, DROP, TRUNCATE).

Q: What is DML (Data Manipulation Language)?

Answer: Commands used to manipulate the data itself. Not auto-committed. (INSERT, UPDATE, DELETE).

Q: What is DQL (Data Query Language)?

Answer: Commands used to fetch data from the database. (SELECT).

Q: What is DCL (Data Control Language)?

Answer: Commands used to manage access rights and permissions. (GRANT, REVOKE).

Q: What is TCL (Transaction Control Language)?

Answer: Commands used to manage transactions. (COMMIT, ROLLBACK, SAVEPOINT).

Q: TRUNCATE vs. DELETE?

Answer: DELETE is a DML command; it removes rows one by one, logs each deletion, and can be rolled back. TRUNCATE is a DDL command; it deallocates the data pages entirely, is much faster, and generally cannot be rolled back.

Q: DROP vs. TRUNCATE?

Answer: TRUNCATE removes all data but keeps the table structure intact. DROP deletes both the data and the entire table structure from the database.

Q: What does the SELECT statement do?

Answer: Retrieves zero or more rows from one or more database tables or views.

Q: How do you fetch unique records?

Answer: By using the SELECT DISTINCT keyword.

Q: What is an Alias?

Answer: A temporary name assigned to a table or column using the AS keyword to make queries more readable.

Q: What is the WHERE clause?

Answer: Used to filter records based on specific conditions before any grouping happens.

Q: WHERE vs. HAVING?

Answer: WHERE filters rows before aggregate functions are calculated. HAVING filters aggregated groups after the GROUP BY clause is applied.

Q: What are Aggregate Functions?

Answer: Functions that perform calculations on a set of values and

return a single scalar value.

: Name the main Aggregate Functions?

Answer: COUNT(), SUM(), AVG(), MAX(), MIN().

Q: What does GROUP BY do?

Answer: Groups rows that have the same values into summary rows, typically used alongside aggregate functions.

Q: What does ORDER BY do?

Answer: Sorts the result set in ascending or descending order.

Q: What is the IN operator?

Answer: Allows you to specify multiple values in a WHERE clause, acting as a shorthand for multiple OR conditions.

Q: What is the BETWEEN operator?

Answer: Filters values within a given range (inclusive of both endpoints).

Q: What is the LIKE operator?

Answer: Used in a WHERE clause to search for a specified pattern in a column using wildcards.

Q: Explain SQL Wildcards?

Answer: The percent sign represents zero, one, or multiple characters. The underscore represents a single character.

Q: What is the EXISTS operator?

Answer: A boolean operator used to test for the existence of any record in a subquery. It evaluates to true if the subquery returns one or more records.

: IN vs. EXISTS?

Answer: IN compares values directly and can be slow for large datasets. EXISTS stops searching the moment it finds a match in the subquery, making it faster for large result sets.

Q: What is the ANY operator?

Answer: Returns true if any of the subquery values meet the condition.

Q: What is the ALL operator?

Answer: Returns true only if all of the subquery values meet the condition.

Q: What is the IS NULL operator?

Answer: Used to check for empty (NULL) values, as you cannot use

standard comparison operators with NULL.

Q: What is COALESCE?

Answer: A function that returns the first non-null expression among its arguments.

Q: What is NULLIF?

Answer: Returns NULL if the two arguments are equal; otherwise, it returns the first argument. Useful for preventing division-by-zero errors.

Q: What is a JOIN in SQL?

Answer: A clause used to combine rows from two or more tables based on a related column between them.

Q: Explain INNER JOIN?

Answer: Returns only the records that have matching values in both tables.

Q: Explain LEFT JOIN?

Answer: Returns all records from the left table, and the matched records from the right table. If no match, the right side returns NULL.

Q: Explain RIGHT JOIN?

Answer: Returns all records from the right table, and the matched records from the left table.

Q: Explain FULL JOIN?

Answer: Returns all records when there is a match in either left or right table. Unmatched rows on either side contain NULLs.

Q: What is a CROSS JOIN?

Answer: Produces the Cartesian product of two tables. If Table A has 5 rows and Table B has 4 rows, the result has 20 rows.

Q: What is a SELF JOIN?

Answer: A regular join, but the table is joined with itself (requires using table aliases).

Q: What is a Subquery?

Answer: A query nested inside another query (SELECT, INSERT, UPDATE, or DELETE).

Q: Types of Subqueries?

Answer: Single-row subqueries, Multi-row subqueries, Multi-column subqueries, and Correlated subqueries.

Q: What is a Correlated Subquery?

Answer: A subquery that depends on the outer query for its values. It executes once for every row processed by the outer query, making it resource-intensive.

Q: Nested Subquery vs. Correlated Subquery?

Answer: A nested subquery can run independently of the outer query (executes once). A correlated subquery cannot run independently.

Q: What are Set Operations?

Answer: Operations that combine the result sets of two or more independent queries into a single result set.

: UNION vs. UNION ALL?

Answer: UNION combines results and removes duplicate rows (incurs a sorting overhead). UNION ALL combines results and keeps all duplicates (much faster).

Q: What is the INTERSECT operator?

Answer: Returns only the distinct rows that are present in both result sets.

: What is the EXCEPT / MINUS operator?

Answer: Returns the distinct rows that are present in the first result set but absent in the second.

Q: What is a View?

Answer: A virtual table based on the result-set of an SQL statement. It doesn't store data physically but provides a way to encapsulate complex queries.

Q: Can you update a View?

Answer: Yes, but only if it's a simple view (based on a single table, no aggregate functions, no GROUP BY, no distinct clauses).

Q: What is a Materialized View?

Answer: Used heavily in data analytics, it physically stores the result of the query on the disk. It drastically improves query time for heavy computations but needs to be refreshed periodically.

: View vs. Materialized View?

Answer: View fetches fresh data every time it's queried (slower). Materialized view reads pre-computed data from disk (faster, but may be slightly stale).

Q: How do you find the Nth highest salary?

Answer: Using ORDER BY salary DESC LIMIT 1 OFFSET N-1 or using window

functions like DENSE_RANK().

Q: How to find duplicate rows in a table?

Answer: `SELECT col, COUNT(*) FROM table GROUP BY col HAVING COUNT(*) > 1;`

Q: How to delete duplicate rows?

Answer: Using a CTE with ROW_NUMBER(): partition by the columns that define duplicates, and delete rows where the row number is > 1.

Q: String functions in SQL?

Answer: CONCAT() to join strings, SUBSTRING() to extract parts, UPPER()/LOWER() for casing, LENGTH() or LEN() for string size.

Q: Date functions in SQL?

Answer: CURRENT_DATE, NOW(), DATEADD(), DATEDIFF(), and EXTRACT() are common for temporal data manipulation.

Q: What is an Identity column / Serial?

Answer: A column whose value is automatically generated by the database upon insertion (e.g., SERIAL in PostgreSQL, AUTO_INCREMENT in MySQL).

Q: What is a Sequence?

Answer: An independent database object that generates a sequence of numeric values according to specific rules, often used for primary keys.

: Sequence vs. Identity Column?

Answer: Identity is tied to a specific table. A Sequence is a standalone object that can be shared across multiple tables.

Q: What is a Synonym?

Answer: An alternative name for database objects like tables, views, or sequences, used to simplify object referencing.

Q: What is a Composite Data Type?

Answer: A data type that allows a single column to hold an entire row of data or a complex JSON object.

Q: Explain the CASE statement?

Answer: SQL's version of an IF-THEN-ELSE logical construct. It iterates through conditions and returns a value when the first condition is met.

Q: What are Transaction States?

Answer: Active, Partially Committed, Failed, Aborted, and Committed.

Q: What is Concurrency Control?

Answer: The mechanism to manage simultaneous execution of transactions in a shared database to ensure data integrity and isolation.

Q: Why is Concurrency Control needed?

Answer: To prevent concurrency anomalies like Lost Updates, Dirty Reads, and Phantom Reads.

Q: What is a Dirty Read?

Answer: Occurs when transaction A reads uncommitted data written by transaction B. If B rolls back, A holds invalid data.

Q: What is a Non-Repeatable Read?

Answer: Occurs when transaction A reads a row, transaction B updates it and commits, and A reads the same row again, seeing different data.

Q: What is a Phantom Read?

Answer: Occurs when transaction A executes a query with a condition (e.g., age > 20), transaction B inserts a new row matching that condition, and A runs the query again, seeing a phantom new row.

Q: What are the Transaction Isolation Levels?

Answer: From least to most strict: Read Uncommitted, Read Committed, Repeatable Read, and Serializable.

Q: What is a Lock?

Answer: A mechanism used by the DBMS to prevent concurrent transactions from interfering with each other.

: Shared Lock (S) vs. Exclusive Lock (X)?

Answer: Shared lock allows a transaction to read but not modify data. Exclusive lock allows a transaction to both read and modify data.

Q: What is Two-Phase Locking (2PL)?

Answer: A protocol that guarantees serializability by dividing a transaction into a Growing Phase (acquiring locks) and a Shrinking Phase (releasing locks).

Q: What is a Deadlock?

Answer: A situation where two transactions wait indefinitely for each other to release locks.

: Deadlock Prevention vs. Detection?

Answer: Prevention ensures a deadlock never occurs (e.g., Wait-Die, Wound-Wait schemes). Detection allows it to happen, but periodically checks for cycles and aborts one transaction.

Q: What is Indexing?

Answer: A data structure technique used to quickly locate and access data in a database, vastly improving query execution speed at the cost of slower writes and increased storage.

: Clustered vs. Non-Clustered Index?

Answer: A Clustered Index dictates the physical order of the data rows on the disk (only 1 per table, usually the PK). A Non-Clustered Index contains pointers to the physical data (you can have multiple per table).

Q: B-Tree vs. Hash Index?

Answer: B-Tree supports range queries and sorting, keeping data balanced. Hash index is extremely fast for exact-match lookups but cannot handle range queries.

Q: What is a Stored Procedure?

Answer: A pre-compiled collection of SQL statements and logic stored inside the database, which can be reused and executed directly by the engine.

: Function vs. Stored Procedure?

Answer: A Function must return a value, can be used in SELECT statements, and generally allows only DQL. A Procedure may or may not return values, allows DML, and is invoked via EXEC or CALL.

Q: What is a Trigger?

Answer: A special type of stored procedure that automatically executes (fires) in response to specific DML events (INSERT, UPDATE, DELETE) on a table.

Q: What are Window Functions?

Answer: Used extensively in data analytics, they perform calculations across a set of related rows without collapsing them into a single row, preserving the original data layout.

: ROW_NUMBER() vs. RANK() vs. DENSE_RANK()?

Answer: ROW_NUMBER gives a sequential integer (1, 2, 3, 4). RANK gives the same number for ties but skips the next (1, 2, 2, 4). DENSE_RANK gives the same number for ties but doesn't skip (1, 2, 2, 3).

Q: What are LEAD() and LAG() functions?

Answer: Analytical functions that access data from a subsequent (LEAD) or preceding (LAG) row in the result set without self-joins.

: What is a CTE (Common Table Expression)?

Answer: A temporary named result set defined using the WITH clause. It makes complex analytical queries highly readable and maintainable

compared to deeply nested subqueries.

Q: Recursive CTE?

Answer: A CTE that references itself. Commonly used to query hierarchical data, like an organizational chart or assembly line sequences.

Q: What does the EXPLAIN command do?

Answer: Returns the execution plan of a query (how the DBMS optimizer intends to execute it, what indexes it uses, etc.). Crucial for identifying performance bottlenecks.

Q: What is Query Optimization?

Answer: The process of choosing the most efficient execution plan for a SQL statement, minimizing CPU, memory, and disk I/O usage.

Q: SQL vs. NoSQL?

Answer: SQL databases are relational, highly structured, use predefined schemas, and enforce ACID vertically. NoSQL databases are non-relational, schema-less, document/key-value oriented, and scale horizontally better for massive unstructured data.

Q: What is the CAP Theorem?

Answer: States that a distributed data store can only guarantee two out of three: Consistency, Availability, and Partition Tolerance. RDBMS favors CA, NoSQL often favors AP or CP.

Q: Sharding vs. Partitioning?

Answer: Partitioning divides a large table into smaller parts within the same database instance. Sharding distributes the data horizontally across multiple independent database servers.

: Vertical vs. Horizontal Partitioning?

Answer: Vertical partitioning splits tables by columns (e.g., separating heavy BLOB columns). Horizontal partitioning splits tables by rows based on a key (e.g., grouping rows by year).

Q: What is a Data Warehouse?

Answer: A central repository of integrated, historical data collected from multiple sources, designed specifically for query, analytics, and business intelligence (OLAP) rather than transactional processing (OLTP).